Terraform

- Terraform is an open source "Infrastructure as a Code" tool, created by Hashi Corp.
- It was developed by Mitchell Hashimoto with Go Language in the year 2014 which All the configuration files used (Hashi Corp Configuration Language) language for the code.
- Terraform uses a simple syntax, can provision infrastructure across multiple clouds & On premises.
- It is Cloud Agnostic it means the systems does not depends on single provider.

TERRAFORM ADVANTAGES:

- Readable code.
- Dry run.
- Importing of Resources is easy.
- Creating of multiple resources. Can create modules for repeatable code.

TERRAFORM DISADVANTAGES:

- Currently under development. Each month, we release a beta version.
- There is no error handling
- There is no way to roll back. As a result, we must delete everything and re-run code.
- A few things are prohibited from import.
- Bugs

Infrastructure as Code (IaC): Terraform is a tool used for implementing Infrastructure as Code. It allows you to define and manage infrastructure configurations in a declarative manner.

Multi-Cloud Support: Terraform is cloud-agnostic and supports multiple cloud providers such as AWS, Azure, Google Cloud, and others. It also works with on-premises and hybrid cloud environments.

Declarative Configuration: Users describe the desired state of their infrastructure in a configuration file (usually written in Hashi Corp Configuration Language - HCL) and Terraform takes care of figuring out how to achieve that state.

Resource Provisioning: Terraform provisions and manages infrastructure resources like virtual machines, storage, networks, and more. It creates and updates resources based on the configuration provided.

State Management: Terraform maintains a state file that keeps track of the current state of the infrastructure. This file is used to plan and apply changes, ensuring that Terraform can update resources accurately.

Plan and Apply Workflow: Before making changes, Terraform generates an execution plan, showing what actions it will take. Users review the plan and then apply it to make the changes to the infrastructure.

Version Control Integration: Terraform configurations can be versioned using version control systems like Git. This allows for collaboration, code review, and tracking changes over time.

Modular Configuration: Infrastructure configurations can be organized into modules, making it easier to reuse and share components across di erent projects.

Community and Ecosystem: Terraform has a vibrant community and a rich ecosystem of modules and providers contributed by the community, making it easier to leverage pre-built solutions for common infrastructure components.

Immutable Infrastructure: Terraform encourages the concept of immutable infrastructure, where changes to infrastructure are made by replacing existing resources rather than modifying them in place.

TERRAFORM COMMANDS:

The Terraform lifecycle refers to the sequence of steps and processes that occur when working with Terraform to manage infrastructure as code. Here's an overview of the typical Terraform lifecycle

Write Configuration: Users define their infrastructure in a declarative configuration language, commonly using HashiCorp Configuration Language (HCL).

Initialize: Run terraform init to initialize a Terraform working directory. This step downloads the necessary providers and sets up the backend.

Plan: Run terraform plan to create an execution plan. Terraform compares the desired state from the configuration with the current state and generates a plan for the changes required to reach the desired state.

Review Plan: Examine the output of the plan to understand what changes Terraform intends to make to the infrastructure. This is an opportunity to verify the planned changes before applying them.

Apply: Execute terraform apply to apply the changes outlined in the plan. Terraform makes the necessary API calls to create, update, or delete resources to align the infrastructure with the desired state

Destroy: When infrastructure is no longer needed, or for testing purposes, run terraform destroy to tear down all resources created by Terraform. This is irreversible, so use with caution.

CREATING EC2 INSTANCE:

```
provider "aws" {
  region     = "ap-south-1"
  access_key = "AKIAWW7WL2JMJKCCMORC"
  secret_key = "DraPAxLZinm+ONtvchniWNG91MpqkwMvy rJVZo/B"
}

resource "aws_instance" "example" {
  ami           = "ami-0af25d0df86db00c1"
  instance_type = "t2.micro"

  tags = {
    name = "web-server"
  }
}
```

TERRAFORM VARIABLE TYPES:

```
resource "aws_instance" "web" {
  ami           = "ami-0af25d0df86db00c1"
  instance_type = var.instance_type

  tags = {
    Name = "web-server"
  }
}
```

```
variable "instance_type" {
  description = "Instance type t2.micro"  # Meaningful description of the variable
  type        = string                    # Variable type (e.g., string, number, list, map, etc.)
  default     = "t2.micro"                # Default value used if not overridden
}
```

- Input Variables serve as parameters for a Terraform module, so users can customize behavior without editing the source.
- Output Values are like return values for a Terraform module. Local Values are a convenience feature for assigning a short name to an expression

TERRAFORM STRING:

- It seems like your question might be incomplete or unclear. If you are looking for information about working with strings in Terraform, I can provide some guidance.
- In Terraform, strings are used to represent text data and can be manipulated using various functions and operators.

```
provider "aws" {
  region     = "ap-south-1"
  access_key = "AKIAWW7WL2JMJKCCMORC"
  secret_key = "DraPAxLZinm+ONtvchniWNG91MpqkwMvy rJVZo/B"
}

resource "aws_instance" "ec2_example" {
  ami           = "ami-0767046d1677be5a0"
  instance_type = var.instance_type

  tags = {
    Name = "Terraform EC2"
  }
}

variable "instance_type" {
  description = "Instance type t2.micro"
  type        = string
  default     = "t2.micro"
}
```

TERRAFORM NUMBER: The number type can represent both whole numbers and fractional values.

```
provider "aws" {
  region     = "ap-south-1"
  access_key = "AKIAWW7WL2JMJKCCMORC"
  secret_key = "DraPAxLZinm+ONtvchniWNG91MpqkwMvy rJVZo/B"
}

resource "aws_instance" "ec2_example" {
  ami           = "ami-0af25d0df86db00c1"
  instance_type = "t2.micro"
  count         = var.instance_count

  tags = {
    Name = "Terraform EC2"
  }
}

variable "instance_count" {
  description = "Instance type count"
  type        = number
  default     = 2
}
```

TERRAFORM BOOLEAN: a Boolean represents a binary value indicating either true or false. Booleans are used to express logical conditions, make decisions, and control the flow of Terraform configurations. In HashiCorp Configuration Language (HCL), which is used for writing Terraform configurations, boolean values are written as true or false.

```
provider "aws" {
   region     = "ap-south-1"
   access_key = "AKIAWW7WL2JMJKCCMORC"
   secret_key = "DraPAxLZinm+ONtvchniWNG91MpqkwMvyrJVZo/B"
}

resource "aws_instance" "ec2_example" {

   ami           = "ami-0af25d0df86db00c1"
   instance_type = "t2.micro"
   count  = 1
   associate_public_ip_address = var.enable_public_ip

   tags = {
           Name = "Terraform EC2"
   }
}

variable "enable_public_ip" {
   description = "Enable public IP"
   type        = bool
   default     = true
}
```

LIST/TUPLE:

```
provider "aws" {
   region     = "ap-south-1"
   access_key = "AKIAWW7WL2JMJKCCMORC"
   secret_key = "DraPAxLZinm+ONtvchniWNG91MpqkwMvyrJVZo/B"
}

resource "aws_instance" "ec2_example" {

   ami           = "ami-0af25d0df86db00c1"
   instance_type = "t2.micro"
   count  = 1

   tags = {
           Name = "Terraform EC2"
   }
}

resource "aws_iam_user" "example" {
  count = length(var.user_names)
  name  = var.user_names[count.index]
}

variable "user_names" {
   description = "IAM USERS"
   type        = list(string)
   default     = ["user1", "user2", "user3"]
}
```

MAP/OBJECT:

```
provider "aws" {
   region     = "ap-south-1"
   access_key = "AKIAWW7WL2JMJKCCMORC"
   secret_key = "DraPAxLZinm+ONtvchniWNG91MpqkwMvyrJVZo/B"
}
resource "aws_instance" "ec2_example" {

   ami           = "ami-0af25d0df86db00c1"
   instance_type =  "t2.micro"

   tags = var.project_environment

}


variable "project_environment" {
  description = "project name and environment"
  type        = map(string)
  default     = {
    project     = "project-alpha",
    environment = "dev"
  }
}
```

LOOPS

FOR LOOP:

The for loop is pretty simple and if you have used any programming language before then I guess you will be pretty much familiar with the for loop.

Only the difference you will notice over here is the syntax in Terraform.

We are going to take the same example by declaring a list(string) and adding three users to it - user1, user2, user3 Use the above ec2 block if you want

```
output "print_the_names" {
  value = [for name in var.user_names : name]
}

variable "user_names" {
  description = "IAM usernames"
  type        = list(string)
  default     = ["user1", "user2", "user3"]
}
```

FOR EACH:

The for each is a little special in terraforming and you cannot use it on any collection variable.

Note: - It can only be used on set(string) or map(string).

The reason why for each does not work on list(string) is because a list can contain duplicate values but if you are using set(string) or map(string) then it does not support duplicate values.

```
resource "aws_iam_user" "example" {
  for_each = var.user_names
  name   = each.value
}

variable "user_names" {
  description = "IAM usernames"
  type        = set(string)
  default     = ["user1", "user2", "user3"]
}
```

LOOPS WITH COUNT:

we need to use count but to use the count first we need to declare collections inside our file.

```
resource "aws_iam_user" "example" {
  count = length(var.user_names)
  name  = var.user_names[count.index]
}

variable "user_names" {
  description = "IAM usernames"
  type        = list(string)
  default     = ["user1", "user2", "user3"]
}
```

TERRAFORM CLI: to pass values form command line during run time

```
provider "aws" {
  region = "ap-south-1"  # Add your desired region
}

# EC2 instance resource using a variable for instance type
resource "aws_instance" "two" {
  ami           = "ami-0715c1897453cabd1"
  instance_type = var.instance_type

  tags = {
    Name = "web-server"
  }
}

# Input variable for instance type
variable "instance_type" {
  description = "EC2 instance type (e.g., t2.micro)"
  type        = string
  default     = "t2.micro"  # Optional: You can override this via tfvars or CLI
}
```

TERRAFORM OUTPUTS: used to show the properties/metadata of resources

```
# AWS provider configuration (region, credentials should be set via profile or env vars)
provider "aws" {
  region = "ap-south-1"  # Add your preferred AWS region
}

# EC2 Instance resource
resource "aws_instance" "two" {
  ami           = "ami-0715c1897453cabd1"
  instance_type = "t2.micro"

  tags = {
    Name = "web-server"
  }
}

# Output block to display instance metadata
output "abc" {
  description = "Public IP, Public DNS, and Private IP of the EC2 instance"
  value = [
    aws_instance.two.public_ip,
    aws_instance.two.public_dns,
    aws_instance.two.private_ip
  ]
}
```

ALIAS & PROVIDERS:

Terraform configuration using multiple AWS providers with aliases to deploy EC2 instances in two different regions: us-east-1 and ap-south-1.

```
# Primary AWS provider (default)
provider "aws" {
  region = "us-east-1"
}

# EC2 instance in us-east-1
resource "aws_instance" "one" {
  ami           = "ami-0715c1897453cabd1"
  instance_type = "t2.micro"

  tags = {
    Name = "web-server"
  }
}

# Secondary AWS provider with alias for ap-south-1
provider "aws" {
  region = "ap-south-1"
  alias  = "south"
}

# EC2 instance in ap-south-1 using aliased provider
resource "aws_instance" "two" {
  provider      = aws.south
  ami           = "ami-0607784b46cbe5816"
  instance_type = "t2.micro"

  tags = {
    Name = "web-server"
  }
```

TERRAFORM WORKSPACE:

In Terraform, a workspace is a way to manage multiple instances of your infrastructure configurations. Workspaces allow you to maintain different sets of infrastructure within the same Terraform configuration files. Each workspace has its own state, variables, and resources, allowing you to manage and deploy distinct environments or configurations.

- Default Workspace: When you initialize a Terraform configuration without explicitly creating a workspace, you are in the default workspace. The default workspace is often used for the main or production environment.
- Create a Workspace: You can create additional workspaces using the **terraform workspace new**
- List Workspaces: To see a list of available workspaces, you can use: **terraform workspace list**
- Select a Workspace: Use the terraform workspace select command to switch between workspaces: **terraform workspace select dev**
- Destroy Specific Workspace: You can destroy resources for a specific workspace using: **terraform workspace select dev && terraform destroy**

HANDS ON WORKSPACE:

```
provider "aws" {
  region     = var.region
  access_key = ""  # Optional: Use environment variables or AWS profiles instead
  secret_key = ""
}

# EC2 Instance resource using input variables
resource "aws_instance" "example" {
  ami           = var.ami
  instance_type = var.size

  tags = {
    Name = "demo-server"
  }
}

# Variable declarations
variable "region" {
  description = "AWS region to deploy the instance"
  default     = "us-east-1"
}

variable "ami" {
  description = "AMI ID for the EC2 instance"
  default     = "ami-00b8917ae86a424c9"
}

variable "size" {
  description = "Instance type"
  default     = "t2.micro"
}
```

This will create instance in default workspace

To Create new Workspace : **terraform workspace new <workspace_name>**

NOTE: we need follow below steps to create resource in new workspace

terraform init

terraform plan

terraform apply

TERRAFORM CODE TO CREATE S3 BUCKET:

```
# S3 Bucket Resource
resource "aws_s3_bucket" "one" {
  bucket = "my-bucket-name"
}

# Ownership Controls
resource "aws_s3_bucket_ownership_controls" "two" {
  bucket = aws_s3_bucket.one.id

  rule {
    object_ownership = "BucketOwnerPreferred"
  }
}

# Access Control List (ACL)
resource "aws_s3_bucket_acl" "three" {
  depends_on = [aws_s3_bucket_ownership_controls.two]
  bucket     = aws_s3_bucket.one.id
  acl        = "private"
}

# Enable Versioning
resource "aws_s3_bucket_versioning" "versioning" {
  bucket = aws_s3_bucket.one.id

  versioning_configuration {
    status = "Enabled"
  }
}
```

STORE ALL THE STATE FILES IN S3 BUCKET:

We can also store state file in S3 bucket, and for that create backend.tf file and add below code in it. Add he below code in main.tf file

```
terraform {
  backend "s3" {
    bucket = "sm7243.ccit"                # Your S3 bucket name (must already exist)
    key    = "prod/terraform.tfstate"     # Path to the state file within the bucket
    region = "us-east-1"                  # AWS region where the S3 bucket is located
  }
}
```

TERRAFORM CODE TO CREATE VPC:

```
# VPC
resource "aws_vpc" "abc" {
  cidr_block           = "10.0.0.0/16"
  instance_tenancy     = "default"
  enable_dns_hostnames = true

  tags = {
    Name = "my-vpc"
  }
}

# Subnet (Public)
resource "aws_subnet" "mysubnet" {
  vpc_id                  = aws_vpc.abc.id
  cidr_block              = "10.0.1.0/24"            # Adjusted for subnet-level granularity
  availability_zone       = "ap-south-1a"
  map_public_ip_on_launch = true                     # Required for public instances

  tags = {
    Name = "subnet-1"
  }
}

# Internet Gateway
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.abc.id

  tags = {
    Name = "my-igw"
  }
}

# Route Table
resource "aws_route_table" "myrt" {
  vpc_id = aws_vpc.abc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "my-route-table"
  }
}

# Route Table Association
resource "aws_route_table_association" "subnet_association" {
  subnet_id      = aws_subnet.mysubnet.id
  route_table_id = aws_route_table.myrt.id
```

TERRAFORM CODE TO CREATE EBS:

```
resource "aws_ebs_volume" "example" {
  availability_zone = "us-west-2a"  # Specify the correct AZ in your region
  size              = 40            # Size in GiB

  tags = {
    Name = "Volume-1"
  }
}
```

TERRAFORM CODE TO CREATE EFS:

```
# AWS Provider
provider "aws" {
  region = "us-east-1"
}

# EFS File System Resource
resource "aws_efs_file_system" "foo" {
  creation_token = "my-product"  # Ensures idempotency across multiple applies

  tags = {
    Name = "swiggy-efs"
  }
}
```

TERRAFORM MODULES:

is a container where you can create multiple resources. Used to create .tf files in the directory structure.

```
.
├── main.tf
├── provider.tf
├── modules
│   ├── instances
│   │   ├── main.tf
│   │   └── variables.tf
│   └── buckets
│       ├── main.tf
│       └── variables.tf
```

provider.tf

```
provider "aws" {
  region = "us-east-1"
}
```

main.tf

```
module "my_instance_module" {
  source         = "./modules/instances"
  ami            = "ami-0a2457eba250ca23d"
  instance_type  = "t2.micro"
  instance_name  = "rahaminstance"
}

module "s3_module" {
  source       = "./modules/buckets"
  bucket_name  = "rahamshaik009988"
}
```

modules/instances/main.tf

```
resource "aws_instance" "my_instance" {
  ami           = var.ami
  instance_type = var.instance_type

  tags = {
    Name = var.instance_name
  }
}
```

modules/instances/variables.tf

```
variable "ami" {
  type = string
}

variable "instance_type" {
  type = string
}

variable "instance_name" {
  description = "Value of the Name tag for the EC2 instance"
  type        = string
}
```

modules/buckets/main.tf

```
resource "aws_s3_bucket" "b" {
  bucket = var.bucket_name
}
```

modules/buckets/variables.tf

```
variable "bucket_name" {
  type = string
}
```

How to Run This

- terraform init
- terraform plan
- terraform apply

IMPORTING EXISITING RESOURCES TO TERRAFORM:

If the infra is already created manually, we can import all the data in terraform.

1. Create provider.tf

```
provider "aws" {
  region  = "us-east-1"
  profile = "default"  # Use the AWS CLI profile configured via `aws configure`
}
```

2. Run Terraform Initialization - terraform init

3. Create main.tf

```
resource "aws_instance" "my-server" {
  ami           = "ami-0578f2b35d0328762"  # Replace with the AMI ID of your existing instance
  instance_type = "t2.micro"               # Match the type of your existing instance

  tags = {
    Name = "my-server"
  }
}
```

4. Run Import Command - terraform import aws_instance.my-server i-0123456789abcdef0

5. Verify the Import - terraform state show aws_instance.my-server

TERRAFORM CODE FOR ASG, VCP, SUBNETS, RT, IGW, SECURITY GROUP & EC2

provider.tf

```
provider "aws" {
  region = "us-east-1"
}
```

vpc.tf

```
resource "aws_vpc" "abc" {
  cidr_block           = "10.0.0.0/16"
  instance_tenancy     = "default"
  enable_dns_hostnames = true

  tags = {
    Name = "my-vpc"
  }
}
```

subnets.tf

```
resource "aws_subnet" "mysubnet1" {
  vpc_id                  = aws_vpc.abc.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "us-east-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "subnet-1"
  }
}

resource "aws_subnet" "mysubnet2" {
  vpc_id                  = aws_vpc.abc.id
  cidr_block              = "10.0.2.0/24"
  availability_zone       = "us-east-1b"
  map_public_ip_on_launch = true

  tags = {
    Name = "subnet-2"
  }
}
```

igw.tf

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.abc.id

  tags = {
    Name = "my-igw"
  }
}
```

rt.tf

```
resource "aws_route_table" "myrt" {
  vpc_id = aws_vpc.abc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "my-route-table"
  }
}

resource "aws_route_table_association" "public_subnet_association_1a" {
  subnet_id      = aws_subnet.mysubnet1.id
  route_table_id = aws_route_table.myrt.id
}

resource "aws_route_table_association" "public_subnet_association_1b" {
  subnet_id      = aws_subnet.mysubnet2.id
  route_table_id = aws_route_table.myrt.id
}
```

security.tf

```
resource "aws_security_group" "web_server" {
  name_prefix = "web-server-sg"
  vpc_id      = aws_vpc.abc.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

main.tf

```
resource "aws_launch_template" "mylt" {

 name                = "terraform-lt"

 description          = "v1"

 image_id            = "ami-0ddc798b3f1a5117e"

 instance_type        = "t2.micro"

 key_name            = "mustafakey-pair"

 vpc_security_group_ids = [aws_security_group.web_server.id]



 placement {
```

```
  availability_zone = "us-east-1a"
 }

 user_data = <<-EOF
        #!/bin/bash
        sudo yum update -y
        sudo yum install httpd -y
        sudo systemctl start httpd
        sudo systemctl enable httpd
        sudo systemctl restart httpd
        sudo chmod 766 /var/www/html/index.html
        echo "<html><body><h1>Welcome to Terraform Scaling.</h1></body></html>" >
/var/www/html/index.html
       EOF
}

resource "aws_elb" "web_server_lb" {
 name            = "web-server-lb"
 security_groups   = [aws_security_group.web_server.id]
 subnets         = [aws_subnet.mysubnet1.id, aws_subnet.mysubnet2.id]

 listener {
  instance_port     = 8000
  instance_protocol = "http"
  lb_port         = 80
  lb_protocol      = "http"
```

```
  }

  tags = {
    Name = "terraform-elb"
  }
}

resource "aws_autoscaling_group" "myasg" {
  name                = "terraform-asg"
  min_size            = 2
  max_size            = 6
  desired_capacity    = 2
  health_check_type   = "EC2"
  load_balancers      = [aws_elb.web_server_lb.name]
  vpc_zone_identifier = [aws_subnet.mysubnet1.id, aws_subnet.mysubnet2.id]

  launch_template {
    id = aws_launch_template.mylt.id
  }

  tag {
    key                 = "Name"
    value               = "web-asg-instance"
    propagate_at_launch = true
  }
}
```

How to Deploy

- terraform init
- terraform plan
- terraform apply

TERRAFORM REFRESH:

terraform refresh is a command used to **update the Terraform state file** to match the **real-world infrastructure** without modifying the actual resources.

It helps Terraform re-sync with what's currently deployed in your cloud environment.

Command Syntax - terraform refresh

TERRAFORM PROVISIONERS:

Terraform provisioners are used to perform custom actions and tasks on local or remote machines during resource creation or destruction. They can be used to run shell scripts, copy f iles, or install software. There are two types of provisioners:

generic provisioners (file, local exec, and remotes-exec) and **vendor provisioners** (chef, habitat, puppet, salt-masterless).

1. **Local-exec Provisioner** This provisioner executes a command on the machine from where terraform code is run. The above code will create a file called instance-id.txt file on our terraform server that contains new created instance id.

2. **Resource Provisioner** This is similar to a local provisioner, which will be run on the machine where terraform is running. The only difference is that it will be run at the end of the terraform execution, while a local provisioner will be run for each resource after resource creation.

3. **Remote-exec Provisioner** This provisioner connects to a remote machine and executes commands inside that machine. This can be helpful to configure software or install packages on the remote machine

4. **File Provisioner** Helps to copy file from local machine to the remote machine, helpful for copying configuration files etc.

5. **Null Resource Provisioner** This provisioner does not create any resource, but can be used to trigger local-exec or remote-exec based on a condition. A common use case for the null_resource provisioner is to execute a local script or command after creating an AWS EC2 instance. In the below example after creating an aws_instance, we use null_resource to perform a local-exec provisioner. The depends_on parameter ensures that the null_resource is created only after aws_instance resource has been successfully created. The null_resource is not a real resource and it is used just to perform a provisioner based on a dependency.

TERRAFORM TAINT:

This command will recreate the infra in same workspace

- **terraform taint aws_instance.one**
- **terraform apply --auto-approve**
- To remove the resource from taint : **terraform untaint aws_instance.one**

TERRAFORM REPLACE:

To recreate any service we will use terraform taint, but as per the new update instead of taint we will directly use terraform replace command.

terraform apply --auto-approve -replace="aws_instance.three"

TERRAFORM LIFECYCLE:

1. PREVENT DESTROY: When we create a resource on Terraform if we apply terraform destroy it will delete automatically but if you want to prevent a resource to without deleting we can use prevent destroy in life cycle**.**

2.create_before_destroy : By default when you modify any argument in Terraform and run apply it will destroy the existing resource and then it will create the new resource but if we put create_before_destroy that means the new resource will be created first and the old resource is going to delete later.

3.IGNORE CHANGES: When we create a resource with the help of Terraform it will convert the desired set into current state if we modify anything on current state then if you run terraform apply or plan it will reflect the changes if you don't want to reflect the changes if you want to ignore them we can use this one

```
resource "aws_instance" "three" {
  ami           = "ami-0d8667b0f72471655"
  instance_type = "t2.micro"
  tags = {
    Name = "abc-server"
  }
  lifecycle {
    ignore_changes = [tags]
  }
}
```

TERRAFORM LOCALS

If we need to use common values in repeated blocks then we go for local blocks.all the values can be declared in a block called locals{}.

```
locals {
  common_tags = {
    users = "devteam"
  }
}
resource "aws_instance" "myec2" {
  ami           = "ami-96fda3c22c1c990a"
  instance_type = "t2.small"
  tags          = local.common_tags
}
resource "aws_instance" "myec3" {
  ami           = "ami-96fda3c22c1c990a"
  instance_type = "t2.small"
  tags          = local.common_tags
}
```

TERRAFORM DYNAMIC BLOCKS: Dynamic blocks helps to reduce repetitive codes.

```
resource "aws_security_group" "mysec"
  name = "prodsecgroups"

  dynamic "ingress" {
    for_each = var.ports
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["10.0.0.0/0"]
    }
  }
}

variable "ports" {
  type    = list(any)
  default = [22, 2049, 80]
}
```